

primitive job for computing gradients in AdaptVQE fails when using Aer's

<https://github.com/Qiskit/qiskit/issues/9137>

ROW 109

primitive job for computing gradients in AdaptVQE fails when using Aer's Estimator with approximation=True #9137

New issue

Closed as not planned



JoelHBierman opened on Nov 15, 2022 · edited by JoelHBierman

Edits ...

Environment

- Qiskit Terra version: 0.22.2 (and Aer version 0.11.1)
- Python version: 3.9.5
- Operating system: MacOS/ Linux Mint

What is happening?

When using Aer's implementation of `Estimator` and setting `approximation=True`, the primitive job for computing the commutator expectation values for the gradients in AdaptVQE fails. This estimator seems to work fine for regular VQE and also for AdaptVQE when `approximation=True` is omitted, which tells me that the issue is specifically related to the AdaptVQE implementation of the gradient calculation.

How can we reproduce the issue?

The following program for calculating the ground state of diatomic hydrogen:

```
from qiskit.algorithms.minimum_eigensolvers import AdaptVQE, VQE
from qiskit_aer.primitives import Estimator
from qiskit_nature.problems.second_quantization import ElectronicStructureProblem
from qiskit_nature.converters.second_quantization import QubitConverter
from qiskit_nature.mappers.second_quantization import JordanWignerMapper
from qiskit_nature.circuit.library import UCCSD, HartreeFock
from qiskit_algorithms.optimizers import COBYLA
import numpy as np

basis = 'sto-3g'
interatomic_distance = 0.735

from qiskit_nature.drivers.second_quantization import ElectronicStructureMoleculeDriver, ElectronicStructureDriver
from qiskit_nature.drivers import Molecule
molecule = Molecule(
    geometry=[["H", [0.0, 0.0, 0.0]], ["H", [0.0, 0.0, interatomic_distance]]], charge=0, multiplicity=1
)
driver = ElectronicStructureMoleculeDriver(
    molecule, basis=basis, driver_type=ElectronicStructureDriverType.PYSFC
)
q_molecule = driver.run()
qubit_converter = QubitConverter(JordanWignerMapper())
es_problem = ElectronicStructureProblem(driver=driver)
second_q_op = es_problem.second_q_ops()
qubit_op = qubit_converter.convert(second_q_op[0])

num_qubits = es_problem.num_spin_orbitals

HF_state = HartreeFock(num_spin_orbitals=es_problem.num_spin_orbitals,
                       num_particles=es_problem.num_particles,
                       qubit_converter=qubit_converter)

ansatz = UCCSD(qubit_converter=qubit_converter,
               num_particles=es_problem.num_particles,
               num_spin_orbitals=es_problem.num_spin_orbitals,
               initial_state=HF_state)

VQE_instance = VQE(ansatz=ansatz,
                   optimizer=COBYLA(),
                   estimator=Estimator(approximation=True),
                   initial_point=np.zeros(ansatz.num_parameters),
                   callback=None)

AdaptVQE_instance = AdaptVQE(VQE_instance)

result = AdaptVQE_instance.compute_minimum_eigenvalue(qubit_op)
print(result)
print(result.optimal_circuit.decompose())
AdaptVQE_instance.ansatz = result.optimal_circuit
print(AdaptVQE_instance.ansatz.decompose())
```

Assignees

ikkoham

Labels

bug mod: algorithms mod: primitives

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



results in the following traceback:

```
Traceback (most recent call last):
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/site-packages/qiskit/estimator.py", line 100, in _run
    expectation_values = estimator.job.result().values
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/site-packages/qiskit/estimator.py", line 100, in _run
    return self._future.result()
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/concurrent/futures/_base.py", line 439, in result
    return self.__get_result()
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/concurrent/futures/_base.py", line 391, in __get_result
    raise self._exception
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/concurrent/futures/_base.py", line 275, in result
    result = self.fn(*self.args, **self.kwargs)
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/site-packages/qiskit/estimator.py", line 100, in _run
    return self._compute_with_approximation(
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/site-packages/qiskit/estimator.py", line 100, in _run
    circuit.save_expectation_value(observable, self._layouts[i])
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/site-packages/qiskit/estimator.py", line 100, in _run
    instr = SaveExpectationValue(operator,
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/site-packages/qiskit/estimator.py", line 100, in _run
    raise ExtensionError("Input operator is not Hermitian.")
qiskit.extensions.exceptions.ExtensionError: 'Input operator is not Hermitian.'
```

The above exception was the direct cause of the following exception:

```
Traceback (most recent call last):
  File "/Users/joelbierman/Desktop/electronic-structure-methods/test/adaptVQE/H2_AdaptVQE_bug.py", line 46, in
    result = AdaptVQE_instance.compute_minimum_eigenvalue(qubit_op)
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/site-packages/qiskit/estimator.py", line 100, in _run
    cur_grads = self._compute_gradients(theta, operator)
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/site-packages/qiskit/estimator.py", line 100, in _run
    res = estimate_observables(self.solver.estimator, self.solver.ansatz, commutators, theta)
  File "/Users/joelbierman/opt/anaconda3/envs/Qiskit-0392-esm-010-nature-045/lib/python3.9/site-packages/qiskit/estimator.py", line 100, in _run
    raise AlgorithmError("The primitive job failed!") from exc
qiskit.algorithms.exceptions.AlgorithmError: 'The primitive job failed!'
```

What should happen?

The algorithm completes without primitive job failure. Aer seems to be flagging these commutator operators as non-Hermitian, which is a bit odd considering that using Aer's Estimator implementation without setting `approximation=True` works just fine. The hermiticity should be independent of this argument.

Any suggestions?

No response



JoelHBierman added **bug** on Nov 15, 2022



woodsp-ibm added **mod: algorithms** **mod: primitives** on Nov 15, 2022



woodsp-ibm assigned [ikkoham](#) on Nov 15, 2022



woodsp-ibm on Nov 15, 2022

Member ...

This may be more an issue for Qiskit Aer repo rather than Terra - [@ikkoham](#) may be able to ascertain better.



JoelHBierman on Nov 17, 2022

Author ...

I see, thank you. Should I close this and re-open it in Aer?



 **woodsp-ibm** assigned [ikkoham](#) on Nov 15, 2022



woodsp-ibm on Nov 15, 2022

Member ...

This may be more an issue for Qiskit Aer repo rather than Terra - [@ikkoham](#) may be able to ascertain better.



JoelHBierman on Nov 17, 2022

Author ...

I see, thank you. Should I close this and re-open it in Aer?



woodsp-ibm on Nov 17, 2022

Member ...

If it is an Aer issue it can be transferred over there. Maybe let [@ikkoham](#) make a determination since he is working with the Aer primitives too and we can go from there.



JoelHBierman on Dec 10, 2022

Author ...

I think I have tracked down where the error occurs and it looks to be an Aer issue, so I can close this and re-open the issue in Aer if you would like.

This error only occurs for Aer's `Estimator` when `approximation` is `True` and `shots` is `None`. Setting `Approximation=True`, but setting `shots` to be some finite integer works just fine. In the code, there is a clear fork in the road that determines whether or not this error pops up in the `_compute_with_approximation` function in the file `/qiskit-aer/primitives/estimator.py`. On line 292, there is the conditional statement:

```
if shots is None:
    circuit.save_expectation_value(observable, self._layouts[i])
else:
    for term_ind, pauli in enumerate(observable.paulis):
        circuit.save_expectation_value(pauli, self._layouts[i], label=str(term_ind))
```



Running `circuit.save_expectation_value` involves creating an instance of `SaveExpectationValue` in the file `qiskit-aer/library/save_instructions/save_expectation_value.py`, which involves checking that the operator is Hermitian. In this case, the gradient operators required by `AdaptVQE` is a `SparsePauliOp` where some of the Pauli coefficients are complex. Instantiating `SaveExpectationValue` involves checking that all of the Pauli coefficients are close to real. If they are not, it throws an error that not all of the Paulis in `SparsePauliOp` are Hermitian.

Now in the case `shots` is not `None`, instead of saving the expectation value of the `SparsePauliOp` as a whole, the code iterates over the individual Pauli terms and saves the expectation values of the individual terms, which neglects the coefficients which are added in later.

I guess one quick fix would be to just save all expectation values as per the second method, regardless of whether or not `shots` is `None`, but I am guessing that given that this conditional statement exists, there are good reasons for not doing it that way. (e.g. performance if `save_expectation_value` incurs a computation overhead every time it is called- might not be desirable to do if not absolutely necessary.)



JoelHBierman on Dec 10, 2022 · edited by JoelHBierman

Edits ▾ Author ...

I guess for now, if you wanted to run noiseless `AdaptVQE` simulations using Aer, the thing to do would be to set `approximation=True`, but make the number of shots really large to make the variance effectively zero since it is just sampling from a gaussian and not actually running the circuit shots individually as when `Approximation=False`. The performance does not seem to get much worse if for instance I set `shots=10**30`.



Microso

This error only occurs for Aer's Estimator when `approximation` is `True` and `shots` is `None`. Setting `Approximation=True`, but setting `shots` to be some finite integer works just fine. In the code, there is a clear fork in the road that determines whether or not this error pops up in the `_compute_with_approximation` function in the file `/qiskit_aer/primitives/estimator.py`. On line 292, there is the conditional statement:

```
if shots is None:
    circuit.save_expectation_value(observable, self._layouts[I])
else:
    for term_ind, pauli in enumerate(observable.paulis):
        circuit.save_expectation_value(pauli, self._layouts[i], label=str(term_ind))
```

Running `circuit.save_expectation_value` involves creating an instance of `SaveExpectationValue` in the file `qiskit_aer/library/save_instructions/save_expectation_value.py`, which involves checking that the operator is Hermitian. In this case, the gradient operators required by `AdaptVQE` is a `SparsePauliOp` where some of the Pauli coefficients are complex. Instantiating `SaveExpectationValue` involves checking that all of the Pauli coefficients are close to real. If they are not, it throws an error that not all of the Paulis in `SparsePauliOp` are Hermitian.

Now in the case `shots` is not `None`, instead of saving the expectation value of the `SparsePauliOp` as a whole, the code iterates over the individual Pauli terms and saves the expectation values of the individual terms, which neglects the coefficients which are added in later.

I guess one quick fix would be to just save all expectation values as per the second method, regardless of whether or not `shots` is `None`, but I am guessing that given that this conditional statement exists, there are good reasons for not doing it that way. (e.g. performance if `save_expectation_value` incurs a computation overhead every time it is called- might not be desirable to do if not absolutely necessary.)



JoelHBierman on Dec 10, 2022 · edited by JoelHBierman

Edits ▾ Author ...

I guess for now, if you wanted to run noiseless `AdaptVQE` simulations using Aer, the thing to do would be to set `approximation=True`, but make the number of shots really large to make the variance effectively zero since it is just sampling from a gaussian and not actually running the circuit shots individually as when `Approximation=False`. The performance does not seem to get much worse if for instance I set `shots=10**30`



woodsp-ibm on Jul 11, 2023

Member ...

I think as far as algorithms go this was resolved so I am going to close this. If there is still some aspect with Aer Estimator feel free to open an issue on Aer for that.



woodsp-ibm closed this as not planned on Jul 11, 2023